

TEMA N° 3



TEMA 3 DISEÑO LÓGICO Y NORMALIZACIÓN



EN ESTE TEMA SERÁS CAPAZ DE...

1. Transformar modelos conceptuales (Diagramas Entidad-Relación) en Modelos Relacionales (tablas) listos para ser implementados en un gestor de bases de datos.
2. Aplicar las reglas de normalización (desde la 1FN hasta la BCNF) para eliminar la redundancia de datos y prevenir anomalías en la gestión de información.
3. Diseñar bases de datos eficientes, íntegras y escalables, orientadas a resolver problemas reales de nuestra comunidad en Pailón, específicamente para el CEA Herman Ortiz Camargo.



PRÁCTICA



¿Alguna vez has notado cómo un registro repetido o mal guardado puede causar un verdadero caos en la administración de estudiantes?

Imagina la siguiente situación real: Nuestro querido CEA Herman Ortiz Camargo acaba de inaugurar su propio módulo educativo, completamente nuevo y bien equipado, ubicado en el Barrio Saó. La dirección ha decidido digitalizar toda la información de los estudiantes. Actualmente, la secretaria maneja todo en una enorme hoja de cálculo (Excel).

En esta hoja, los estudiantes de Sistemas Informáticos que viven en el Barrio Central Fátima, Barrio 24 de Septiembre y Barrio Residencial Norte están mezclados en una sola lista gigante. Cada fila tiene el nombre del estudiante, su barrio, el nombre de la carrera, el nombre del profesor y el aula.

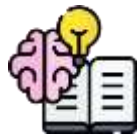
Ayer ocurrieron tres desastres administrativos:

1. Un estudiante que vive en el Barrio 27 de Mayo cambió su número de celular. La secretaria tuvo que buscar manualmente su nombre en 15 filas diferentes (porque está inscrito en varios módulos) para actualizar el número. Se olvidó de actualizar una fila, y ahora el sistema tiene dos números diferentes para la misma persona.
2. El CEA decidió dejar de dictar temporalmente un módulo de ofimática porque el profesor renunció. Al borrar a todos los alumnos de esa materia, ¡también se borró por accidente la información de que el estudiante Juan Pérez, residente del barrio Las Misiones, existía en la institución!
3. Se quiso registrar un nuevo módulo de "Redes", pero como aún no hay estudiantes inscritos de San Antonio ni de Jardines de Pailón, el sistema de Excel no permitió guardar la información de la materia sola.

Como futuro Técnico en Sistemas Informáticos, te enfrentas a un reto ineludible: los datos están "sucios", redundantes y son imposibles de mantener. ¿Cómo estructuramos esta información



para que la base de datos sea rápida, segura y no pierda información valiosa de nuestra institución? Aquí es donde entra la magia del Diseño Lógico y la Normalización.



TEORÍA



3.1. CONVERSIÓN DEL DIAGRAMA E-R A MODELO RELACIONAL (TABLAS)

El primer paso de un Técnico en Sistemas Informáticos es llevar el diseño dibujado en papel (Diagrama Entidad-Relación) al lenguaje que entienden las computadoras: el Modelo Relacional. Este modelo se basa en **Tablas** (relaciones).

1. **Entidades a Tablas:** Cada entidad fuerte (como Estudiante o Modulo) se convierte en una tabla.
2. **Atributos a Columnas:** Los atributos (nombre, apellido, CI) pasan a ser las columnas de esa tabla.
3. **Identificadores a Claves Primarias (Primary Keys - PK):** El atributo que identifica de forma única a la entidad (por ejemplo, el Carnet de Identidad o un Código de Estudiante) se convierte en la Clave Primaria. Ningún valor en esta columna puede repetirse.





Ejemplo Práctico:

Si en el papel teníamos la entidad Barrio con los atributos ID_Barrio, Nombre_Barrio y Distancia_al_CEA. En el modelo relacional, esto será una tabla llamada BARRIOS con tres columnas, donde ID_Barrio será la Clave Primaria.

3.2. CLAVES FORÁNEAS (FOREIGN KEYS) E INTEGRIDAD REFERENCIAL

Para que las tablas no sean islas aisladas, deben relacionarse. Aquí nacen las **Claves Foráneas (FK)**. Una clave foránea es una columna en una tabla que hace referencia a la clave primaria de otra tabla.

La **Integridad Referencial** es una regla de oro: "No puedes usar un dato que no existe". Si un estudiante dice vivir en el "Barrio Ovidio Barberi", ese barrio debe existir primero en la tabla de BARRIOS. Si alguien intenta borrar el "Barrio Ovidio Barberi" de la base de datos mientras hay estudiantes viviendo allí, el sistema lo impedirá para proteger la información.

SQL

```
-- Creamos la tabla principal (Padre) para Los barrios de Pailón
CREATE TABLE Barrios (
    -- ID único para cada barrio, Llave primaria
    id_barrio INT PRIMARY KEY,
    -- Nombre del barrio, no puede quedar vacío
    nombre_barrio VARCHAR(50) NOT NULL );

-- Creamos la tabla secundaria (Hija) para Los estudiantes del CEA
CREATE TABLE Estudiantes (
    -- ID único del estudiante, Llave primaria
    id_estudiante INT PRIMARY KEY,
    -- Nombre del estudiante
    nombre VARCHAR(100) NOT NULL,
    -- Clave foránea que conectará con la tabla Barrios
    id_barrio INT,
    -- Establecemos la relación de Integridad Referencial
    FOREIGN KEY (id_barrio) REFERENCES Barrios(id_barrio) );
```

3.3. CONCEPTO DE ANOMALÍAS DE INSERCIÓN, BORRADO Y ACTUALIZACIÓN

Una base de datos mal diseñada, como el Excel de nuestra secretaria en el Momento 1, sufre de anomalías. Estos son errores o comportamientos indeseados que ocurren por tener datos redundantes.

1. **Anomalía de Actualización:** Ocurre cuando un dato (como la dirección del CEA) está repetido en 1000 filas. Si el CEA se muda, debes actualizar 1000 filas. Si te equivocas en una, la base de datos queda inconsistente.



2. **Anomalía de Borrado:** Al borrar un registro, accidentalmente pierdes información sobre otro dato importante. (Ejemplo: Al borrar al único estudiante del Barrio San José, borramos también la existencia de ese barrio en el sistema).
3. **Anomalía de Inserción:** Imposibilidad de guardar un dato porque falta información de otro. (Ejemplo: No poder registrar el barrio "María Belén" en el sistema hasta que al menos un estudiante se inscriba allí).

TABLA NO NORMALIZADA: PROYECTOS Y EMPLEADOS

ID_EMPLEADO	NOMBRE	PROYECTO	UBICACIÓN_PROYECTO	DEPARTAMENTO	DESCRIPCIÓN_DPTO
21	Maria	A	Matstat	Mérciadio	Descripción detamento
22	Santa Ferner	A	San Racino	Departamento	Computentorna
23	Kardiello	A	San Racino	Fundiado	Comoprendários
24	Alelia	A	Proyecto		identarera
25	Julia	A	Proyecto		candirios
26	Canila	A	San Racino	Departamento	Comprendários
27	Andrés	B	San Racino	Departamento	Concripción datamento
28	X	B	Proyecto	Fundiado	Descripción datamento
39	Marco	C	Proyecto	Márcoso	Planentante
40	Mazialo	C	San Racino	Departamento	Contas candirios
		D	San Racino	Departamento	Contas canditos

ANOMALÍA DE ACTUALIZACIÓN: Cambiar la UBICACIÓN del Proyecto 'A' requiere actualizar múltiples filas.

ANOMALÍA DE BORRADO: Si se borra al EMPLEADO 'X', se pierde la información de que el PROYECTO 'B' existía.

ANOMALÍA DE INSERCIÓN: No se puede añadir un nuevo PROYECTO sin tener un EMPLEADO asignado. O para añadir un nuevo DEPARTAMENTO sin empleados.

LA REDUNDANCIA DE DATOS PROVOCA ESTAS ANOMALÍAS. NORMALIZACIÓN

3.4. PRIMERA Y SEGUNDA FORMA NORMAL (1FN, 2FN)

La **Normalización** es el proceso de organizar los datos para reducir la redundancia y eliminar anomalías. Se hace aplicando reglas llamadas "Formas Normales".

Primera Forma Normal (1FN): ¡Atributos Atómicos!

Una tabla está en 1FN si y solo si todos sus atributos son atómicos, es decir, indivisibles. No puedes tener listas o valores múltiples en una sola celda.

1. *Mal:* Estudiante: Juan. Teléfonos: 77711122, 66633344.
2. *Bien:* Dividir en filas distintas o crear una tabla separada para teléfonos.

Segunda Forma Normal (2FN): ¡Dependencia Funcional Completa!



Una tabla está en 2FN si ya está en 1FN y, además, ningún atributo que no sea clave depende de solo una parte de la Clave Primaria (esto aplica cuando la clave primaria está compuesta por dos o más columnas).

1. *Ejemplo:* Si tenemos una tabla Inscripciones con Clave Primaria compuesta de (id_estudiante, id_modulo). Si en esa tabla guardamos el nombre_modulo, ese nombre solo depende de id_modulo (una parte de la clave), no del estudiante. Esto viola la 2FN. El nombre_modulo debe irse a su propia tabla Modulos.

3.5. TERCERA FORMA NORMAL (3FN) Y FORMA NORMAL DE BOYCE-CODD (BCNF)

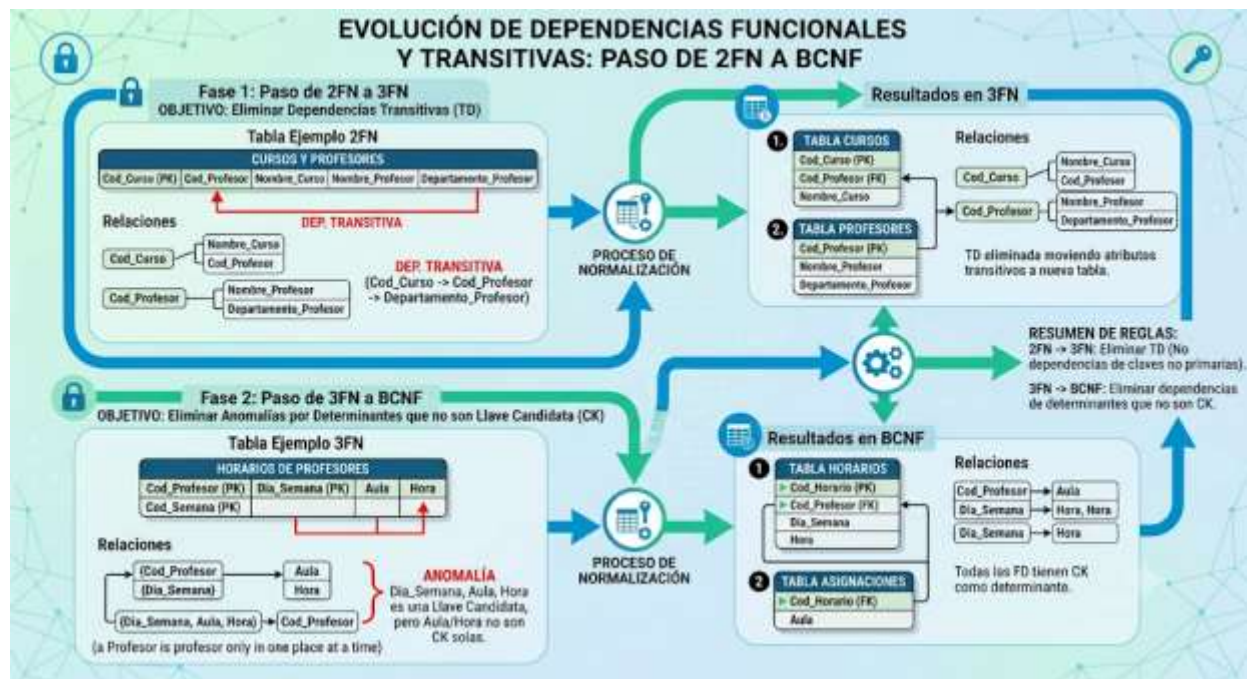
Tercera Forma Normal (3FN): ¡Sin Dependencias Transitivas!

Una tabla está en 3FN si ya está en 2FN y, además, no existen dependencias transitivas. Una dependencia transitiva ocurre cuando un atributo no clave depende de otro atributo no clave. La regla de oro es: *"Cada columna debe depender de la clave, toda la clave y nada más que la clave"*.

1. *Ejemplo:* Tabla Estudiantes con columnas: id_estudiante (PK), nombre, id_barrio, costo_transporte_barrio. El costo del transporte depende del barrio, no directamente del estudiante. ¡Viola la 3FN! Debemos mover costo_transporte_barrio a la tabla Barrios.

Forma Normal de Boyce-Codd (BCNF):

Es una versión más estricta de la 3FN. A veces, en estructuras complejas, una parte de la Clave Primaria puede depender de un atributo que no es clave. La regla de BCNF dice: *"Para cada dependencia funcional $X \rightarrow Y$, X debe ser una clave candidata"*. Para el nivel de Técnico en Sistemas Informáticos, llegar a la 3FN suele ser suficiente para el 95% de los sistemas comerciales, pero BCNF blindará matemáticamente tu base de datos contra redundancias anómalas.



3.6. DISEÑO LÓGICO OPTIMIZADO PARA BASES DE DATOS EN LA NUBE (CLOUD)

Como Técnico, debes saber que el CEA no siempre tendrá sus servidores físicamente en el Barrio Saó. Hoy en día, las bases de datos se alojan en la Nube (AWS, Google Cloud, Azure). Un diseño lógico bien normalizado permite que las consultas (Queries) en la nube sean más rápidas y consuman menos "cómputo". En la nube te cobran por la cantidad de datos que lees y almacenas. Si tu base de datos está normalizada (no repites datos), la base de datos pesa menos, las copias de seguridad son más veloces y la institución ahorra dinero en su facturación mensual.

3.7. IMPACTO DE LA INTELIGENCIA ARTIFICIAL EN EL DISEÑO DE BASES DE DATOS

Actualmente, el Diseño Lógico está evolucionando gracias a la Inteligencia Artificial. Herramientas de IA pueden analizar grandes tablas "sucias" en Excel (como la de nuestra secretaria) y sugerir automáticamente un modelo relacional en 3FN. Como Técnico, tu trabajo ya no será dibujar cajitas a mano durante horas, sino ser el "Arquitecto Supervisor" que valida si la IA comprendió correctamente el contexto del CEA Herman Ortiz Camargo y si las Claves Foráneas que sugiere tienen sentido lógico y de negocio.



3.8. BASES DE DATOS MODERNAS: RELACIONAL VS. NOSQL

Aunque la Normalización es vital para los datos contables, académicos y transaccionales del CEA, en la tecnología actual a veces se "Desnormaliza" a propósito. Para aplicaciones web masivas (como redes sociales), se usan bases de datos NoSQL (como MongoDB) donde la velocidad de lectura es más importante que evitar la redundancia. Sin embargo, para sistemas institucionales cerrados donde la precisión de los datos de los estudiantes y sus notas es crítica, el Modelo Relacional y la 3FN siguen siendo los reyes indiscutibles.

CIERRE TEÓRICO OBLIGATORIO

□ *Mitos vs. Realidades en el Diseño Lógico*

Mito	Realidad
"Normalizar hace que la base de datos sea más lenta"	FALSO. Aunque requiere hacer "JOINS" (uniones), una base normalizada agiliza las actualizaciones, ahorra espacio de disco y mantiene índices mucho más rápidos de leer.
"Tener todo en una sola tabla grande es más fácil de programar"	FALSO. A corto plazo parece fácil, pero al presentarse las anomalías de inserción/borrado, el código de programación se vuelve una pesadilla llena de parches (bugs).

□ *Glosario de Términos*

1. **Modelo Relacional:** Paradigma de organización de bases de datos basado en tablas interconectadas, desarrollado por Edgar Codd.
2. **Clave Primaria (PK):** Columna o conjunto de columnas que identifican unívocamente a cada fila de una tabla.
3. **Clave Foránea (FK):** Columna que establece una relación con la clave primaria de otra tabla, garantizando la integridad referencial.
4. **Redundancia:** Repetición innecesaria de datos dentro de una base de datos, lo cual desperdicia espacio y genera inconsistencias.



5. **Dependencia Transitiva:** Situación donde un atributo que no es clave depende de otro atributo que tampoco es clave primaria. Es el enemigo de la 3FN.

□ *Ponte a Prueba*

1. ¿Qué anomalía ocurre si no puedo registrar la especialidad de "Sistemas Informáticos" porque aún no hay estudiantes inscritos en ella?
 1. a) Anomalía de Actualización
 2. b) Anomalía de Inserción
 3. c) Anomalía de Borrado
2. Si una tabla tiene los teléfonos separados por comas en una sola columna, ¿qué Forma Normal está violando?
 1. a) 1FN
 2. b) 2FN
 3. c) 3FN
3. La Integridad Referencial impide que...
 1. a) Existan claves primarias compuestas.
 2. b) Un estudiante se registre en un barrio que no existe en la base de datos.
 3. c) El disco duro se llene.



VALORACIÓN



Como Técnico en Sistemas Informáticos, tu rol va más allá de escribir código; tienes una responsabilidad social y ética profunda. Al aplicar la normalización en el nuevo módulo del CEA Herman Ortiz Camargo, estás protegiendo el historial académico de personas de Pailón que se esfuerzan por estudiar.



Piensa en la **privacidad y seguridad**: al tener la base de datos centralizada y normalizada, es mucho más fácil encriptar tablas sensibles, evitando que los datos de jóvenes de nuestros barrios caigan en manos equivocadas por culpa de archivos de Excel dispersos.

Desde la **perspectiva medioambiental**, aunque no lo parezca a simple vista, bases de datos redundantes (pesadas) requieren más servidores para almacenamiento y procesamiento de copias de seguridad. Al optimizar los datos, reducimos el consumo de energía eléctrica de los servidores (Data Centers) y el ciclo de reemplazo de discos duros, contribuyendo de manera indirecta, pero real, a la disminución de la basura tecnológica (e-waste). Un código eficiente es un código amigable con el medio ambiente.



PRODUCCIÓN



LABORATORIO PRÁCTICO: "NORMALIZANDO EL REGISTRO DEL CEA HERMAN ORTIZ CAMARGO"

Objetivo: Crear el esquema relacional en SQL para solucionar el problema del Momento 1, alcanzando la 3FN.

Paso 1: Identificar y separar las entidades independientes. Crearemos primero las tablas de catálogos (Barrios y Módulos) para que no haya dependencia transitiva en la información del estudiante.

Paso 2: Crear la tabla central de Estudiantes, enlazándola mediante Claves Foráneas.

Paso 3: Como un estudiante puede tomar muchos módulos, y un módulo tiene muchos estudiantes (Relación Muchos a Muchos), debemos crear una tabla intermedia llamada Inscripciones (Resolución de la 1FN y 2FN).

Abre tu entorno de base de datos (PostgreSQL, MySQL o SQLite) y ejecuta el siguiente script paso a paso:



SQL

```
-- PASO 1: Creación de tablas de catálogo (Eliminando anomalías de inserción)
-- Creamos la tabla de Barrios de Pailón
CREATE TABLE Barrios_Pailon (
    id_barrio INT PRIMARY KEY,
    nombre_barrio VARCHAR(60) NOT NULL );
-- Creamos la tabla de Módulos (Materias) del CEA
CREATE TABLE Modulos_CEA (
    id_modulo INT PRIMARY KEY,
    nombre_modulo VARCHAR(80) NOT NULL,
    nivel_semestral INT NOT NULL );
-- PASO 2: Creación de la tabla Estudiantes (3FN, sin datos transitivos)
CREATE TABLE Estudiantes_CEA (
    ci_estudiante VARCHAR(15) PRIMARY KEY,
    nombres VARCHAR(100) NOT NULL,
    apellidos VARCHAR(100) NOT NULL,
    id_barrio INT,
    FOREIGN KEY (id_barrio) REFERENCES Barrios_Pailon(id_barrio) );
-- PASO 3: Tabla intermedia de Inscripciones (Para cumplir con la 2FN)
-- Evitamos la dependencia parcial al no guardar datos del módulo aquí.
CREATE TABLE Inscripciones (
    id_inscripcion INT PRIMARY KEY,
    ci_estudiante VARCHAR(15),
    id_modulo INT,
    fecha_inscripcion DATE,
    FOREIGN KEY (ci_estudiante) REFERENCES Estudiantes_CEA(ci_estudiante),
    FOREIGN KEY (id_modulo) REFERENCES Modulos_CEA(id_modulo));
-- Felicidades Técnico, has creado una base de datos estructurada,
-- libre de anomalías y lista para el nuevo CEA.
```

RECURSOS ADICIONALES

Para seguir perfeccionando tus habilidades como Técnico en Sistemas Informáticos, te recomiendo explorar las siguientes herramientas oficiales y gratuitas:

1. **Documentación Oficial de PostgreSQL (Tutoriales de Modelado Relacional):** Excelente lugar para profundizar en cómo los grandes motores de base de datos manejan las Foreign Keys y los JOINS. Enlace: <https://www.postgresql.org/docs/>
2. **MDN Web Docs - Introducción a bases de datos y SQL:** Una guía fantástica, amigable y didáctica elaborada por Mozilla, ideal para reforzar los conceptos de estructuración de información.



Enlace: https://developer.mozilla.org/es/docs/Learn/Server-side/First_steps/Database_concepts

3. **dbdiagram.io - Herramienta de Modelado de Base de Datos:** Una aplicación web gratuita para dibujar tus diagramas E-R escribiendo código simple, permitiéndote exportarlos directamente a SQL para que valides tus Formas Normales de forma visual y moderna.

Enlace: <https://dbdiagram.io/>